Summarize

Chapter 12. Resiliency

As software becomes an increasingly vital part of our users' lives, we need to continually improve the quality of service we offer. Software failing can have a significant impact on people's lives, even if the software doesn't fall into the category of "safety critical" in the way that things like aircraft control systems do. During the COVID-19 pandemic, which was ongoing at the time of writing, services like online grocery shopping went from being a convenience to becoming a necessity for many people who were unable to leave their homes.

Against this backdrop, we are often being tasked with creating software that is more and more *reliable*. The expectations of our users have shifted in terms of what the software can do and when it should be available. The days of only having to support software during office hours are increasingly rare, and there is decreasing tolerance for downtime due to maintenance.

As we covered at the start of this book, there is a host of reasons as to why microservice architectures are being chosen by organizations around the world. But for many, the prospects of improving the *resilience* of their service offerings is cited as a major reason.

Before we get into the details of how a microservice architecture can enable resiliency, it's important to step back and consider what resiliency actually is. It turns out that when it comes to improving the resiliency of our software, adopting a microservice architecture is only part of the puzzle.

What Is Resiliency?

We use the term *resiliency* in many different contexts and in many different ways. This can lead to confusion about what the term means and can also result in us thinking too narrowly about the field. Outside the confines of IT, there is a broader area of resilience engineering, which looks at the concept of resiliency as it applies to a host of systems—from firefighting to air traffic control, biological systems, and operating theaters.

Drawing on this field, David D. Woods has attempted to categorize the different aspects of resilience to help us think more widely about what resiliency actually means.[1] These four concepts are:

Robustness

: The ability to absorb expected perturbation

Rebound

: The ability to recover after a traumatic event

Graceful extensibility

: How well we deal with a situation that is unexpected

Sustained adaptability

: The ability to continually adapt to changing environments, stakeholders, and demands

Let's look at each of these concepts in turn and examine how these ideas might (or might not) translate into our world of building microservice architectures.

Robustness

Robustness is the concept whereby we build mechanisms into our software and processes to accommodate expected problems. We have an advanced understanding of the kinds of perturbations we might face, and we put measures in place so that when these issues arise, our system can deal with them. In the context of our microservice architecture, we have a whole host of perturbations that we might expect: a host can fail, a network connection can time out, a microservice might be unavailable. We can improve the robustness of our architecture in a number of ways to deal with these perturbations, such as automatically spinning up a replacement host, performing retries, or handling failure of a given microservice in a graceful manner.

Robustness goes beyond software, though. It can apply to people. If you have a single person on call for your software, what happens if that person gets sick or isn't reachable at the time of an incident? This is a fairly easy thing to consider, and the solution might be to have a backup on-call person.

Robustness by definition requires prior knowledge—we are putting measures into place to deal with known perturbations. This knowledge could be based on foresight: we could

draw on our understandings of the computer system we are building, its supporting services, and our people to consider what might go wrong. But robustness can also come from hindsight—we may improve the robustness of our system after something we didn't expect happens. Perhaps we never considered the fact that our global filesystem could become unavailable, or perhaps we underestimated the impact of our customer service representatives not being available outside working hours.

One of the challenges around improving the robustness of our system is that as we increase the robustness of our application, we introduce more complexity to our system, which can be the source of new issues. Let's say you're moving your microservice architecture to Kubernetes, because you want it to handle desired state management for your microservice workloads. You may have improved some aspects of the robustness of your application as a result, but you've also introduced new potential pain points as well. As such, any attempt to improve the robustness of an application has to be considered, not just in terms of a simple cost/benefit analysis but also in terms of whether or not you're happy with the more complex system you'll have as a result of this.

Robustness is one area in which microservices give you a host of options, and much of what follows in this chapter will focus on what you can do in your software to improve the system's robustness. Just remember that not only is this only one facet of resiliency as a whole, but there is also a host of other nonsoftware-related robustness you might need to consider.

Rebound

How well we recover—rebound—from disruption is a key part of building a resilient system. All too often I see people focusing their time and energy on trying to eliminate the possibility of an outage, only to be totally unprepared once an outage actually occurs. By all means, do your best to protect against the bad things that you think might happen—improving your system's *robustness*—but also understand that as your system grows in scale and complexity, eliminating any potential problem becomes unsustainable.

We can improve our ability to rebound from an incident by putting things into place in advance. For example, having backups in place can allow us to better rebound in the aftermath of data loss (assuming our backups are tested, of course!). Improving our ability to rebound could also include having a playbook we can run through in the wake of a system outage: Do people understand what their role is when an outage occurs? Who will be the point person for handling the situation? How quickly do we need to let our users know what is happening? How will we communicate with our users? Trying to think clearly about how to handle an outage while the outage is going on will be problematic

due to the inherent stress and chaos of the situation. Having an agreed plan of action in place in anticipation of this sort of problem can help you better rebound.

Graceful Extensibility

With rebound and robustness, we are primarily dealing with the expected. We are putting mechanisms in place to deal with problems that we can foresee. But what happens when we are surprised? If we aren't prepared for surprise—for the fact that our expected view of the world might be wrong—we end up with a brittle system. As we approach the limits of what we expect our system to be able to handle, things fall apart—we are unable to perform adequately.

Flatter organizations—where responsibility is distributed into the organization rather than held centrally—will often be better prepared to deal with surprise. When the unexpected occurs, if people are restricted in what they have to do, if they have to adhere to a strict set of rules, their ability to deal with surprise will be critically curtailed.

Often, in a drive to optimize our system, we can as an unfortunate side effect increase the brittleness of our system. Take automation as an example. Automation is fantastic—it allows us to do more with the people we have, but it can also allow us to *reduce* the people we have, as more can be done with automation. This reduction in staff can be concerning, though. Automation can't handle surprise—our ability to gracefully extend our system, to handle surprise, comes from having people in place with the right skills, experience, and responsibility to handle these situations as they arise.

Sustained Adaptability

Having sustained adaptability requires us to not be complacent. As David Woods puts it: "No matter how good we have done before, no matter how successful we've been, the future could be different, and we might not be well adapted. We might be precarious and fragile in the face of that new future."[2] That we haven't yet suffered from a catastrophic outage doesn't mean that it cannot happen. We need to challenge ourselves to make sure we are constantly adapting what we do as an organization to ensure future resiliency. Done right, a concept like chaos engineering—which we'll explore briefly later in this chapter—can be a useful tool in helping build sustained adaptability.

Sustained adaptability often requires a more holistic view of the system. This is, paradoxically, where a drive toward smaller, autonomous teams with increased local, focused responsibility can end with us losing sight of the bigger picture. As we'll explore in Chapter 15, there is a balancing act between global and local optimization when it

comes to organizational dynamics, and that balance isn't static. In that chapter we'll look at the role of focused, stream-aligned teams who own the microservices they need to deliver user-facing functionality and have increased levels of responsibility to make this happen. We'll also look at the role of enabling teams, who support these stream-aligned teams in doing their work, and at how enabling teams can be a big part of helping achieve sustained adaptability at an organizational level.

Creating a culture that prioritizes creating an environment in which people can share information freely, without fear of retribution, is vital to encourage learning in the wake of an incident. Having the bandwidth to really examine such surprises and extract the key learnings requires time, energy, and people—all things that will reduce the resources available to you to deliver features in the short term. Deciding to embrace sustained adaptability is partly about finding the balancing point between short-term delivery and longer-term adaptability.

To work toward sustained adaptability means that you are looking to discover what you don't know. This requires continuing investment, not one-off transactional activities—the term *sustained* is important here. It's about making sustained adaptability a core part of your organizational strategy and culture.

And Microservice Architecture

As we've discussed, we can see a way in which a microservice architecture can help us achieve the property of robustness, but it is not enough if you want *resiliency*.

Taken more broadly, the ability to deliver resiliency is a property not of the software itself but of the people building and running the system. Given the focus of this book, much of what follows in this chapter will focus primarily on what a microservice architecture can help deliver in terms of resiliency—which is almost entirely limited to improving the *robustness* of applications.

Failure Is Everywhere

We understand that things can go wrong. Hard disks can fail. Our software can crash. And as anyone who has read the fallacies of distributed computing can tell you, the network is unreliable. We can do our best to try to limit the causes of failure, but at a certain scale, failure becomes inevitable. Hard drives, for example, are more reliable now than ever before, but they'll break eventually. The more hard drives you have, the higher

the likelihood of failure for an individual unit on any given day; failure becomes a statistical certainty at scale.

Even for those of us not thinking at extreme scale, if we can embrace the possibility of failure we will be better off. For example, if we can handle the failure of a microservice gracefully, then it follows that we can also do in-place upgrades of a service, as a planned outage is much easier to deal with than an unplanned one.

We can also spend a bit less of our time trying to stop the inevitable and a bit more of our time dealing with it gracefully. I'm amazed at how many organizations put processes and controls in place to try to stop failure from occurring but put little to no thought into actually making it easier to recover from failure in the first place. Understanding the things that are likely to fail is key to improving the *robustness* of our system.

Baking in the assumption that everything can and will fail leads you to think differently about how you solve problems. Remember the story of the Google servers we discussed in Chapter 10? The Google systems were built in such a way that if a machine failed, it would not lead to interruption of service—improving the *robustness* of the system as a whole. Google goes further in attempting to improve the robustness of its servers in other ways—it has discussed how each server contains its own local power supply to ensure it can keep operating if the data center has an outage.[3] As you'll recall from Chapter 10, the hard drives in these servers were attached with Velcro rather than screws to make it easy to replace drives—helping Google get the machine up and running quickly when a drive failed, and in turn helping that component of the system *rebound* more effectively.

So let me repeat: at scale, even if you buy the best kit, the most expensive hardware, you cannot avoid the fact that things can and will fail. Therefore, you need to assume failure can happen. If you build this thinking into everything you do and plan for failure, you can make informed trade-offs. If you know your system can handle the fact that a server can and will fail, there may be diminishing returns from spending more and more money on individual machines. Instead, having a larger number of cheaper machines (perhaps using cheaper components and some Velcro!) like Google did may make a lot more sense.

How Much Is Too Much?

We touched on the topic of cross-functional requirements in Chapter 9. Understanding cross-functional requirements is all about considering aspects like durability of data,

availability of services, throughput, and acceptable latency of operations. Many of the techniques covered in this chapter talk about approaches to implement these requirements, but only you know exactly what the requirements themselves might be. So keep your own requirements in mind as you read on.

Having an autoscaling system capable of reacting to increased load or the failure of individual nodes might be fantastic, but it could be overkill for a reporting system that needs to run only twice a month, where being down for a day or two isn't that big of a deal. Likewise, figuring out how to do zero-downtime deployments to eliminate interruption of service might make sense for your online ecommerce system, but for your corporate intranet knowledge base, it's probably a step too far.

How much failure you can tolerate or how fast your system needs to be is driven by the users of your system. That information in turn helps you understand which techniques will make the most sense for you. That said, your users won't always be able to articulate what their exact requirements are. So you need to ask questions to help extract the right information and help them understand the relative costs of providing different levels of service.

As I mentioned previously, these cross-functional requirements can vary from service to service, but I would suggest defining some general cross-functionals and then overriding them for particular use cases. When it comes to considering if and how to scale out your system to better handle load or failure, start by trying to understand the following requirements:

Response time/latency

How long should various operations take? It can be useful to measure this with different numbers of users to understand how increasing load will impact the response time. Given the nature of networks, you'll always have outliers, so setting targets for a given percentile of the responses monitored can be useful. The target should also include the number of concurrent connections/users you will expect your software to handle. So you might say, "We expect the website to have a 90th-percentile response time of 2 seconds when handling 200 concurrent connections per second."

Availability

Can you expect a service to be down? Is this considered a 24/7 service? Some people like to look at periods of acceptable downtime when measuring availability, but how useful is this to someone calling your service? Either I should be able to

rely on your service responding or I shouldn't. Measuring periods of downtime is really more useful from a historical reporting angle.

Durability of data

How much data loss is acceptable? How long should data be kept for? This is highly likely to change on a case-by-case basis. For example, you might choose to keep user session logs for a year or less to save space, but your financial transaction records might need to be kept for many years.

Taking these ideas and articulating them as service-level objectives (SLOs), which we covered in Chapter 10, can be a good way to enshrine these requirements as a core part of your software delivery process.

Degrading Functionality

An essential part of building a resilient system, especially when your functionality is spread over a number of different microservices that may be up or down, is the ability to safely degrade functionality. Let's imagine a standard web page on our ecommerce site. To pull together the various parts of that website, we might need several microservices to play a part. One microservice might display the details about the item being offered for sale. Another might show the price and stock level. And we'll probably be showing shopping cart contents too, which may be yet another microservice. If one of those services goes down, and that results in the whole web page being unavailable, then we have arguably made a system that is less resilient than one that requires only one service to be available.

What we need to do is understand the impact of each outage and work out how to properly degrade functionality. From a business point of view, we would want our order-taking workflow to be as robust as possible, and we might be happy to accept some degradation of functionality to ensure this still works. If the stock levels are unavailable, we might make the decision to still go ahead with the sale and work out the details later. If the shopping cart microservice is unavailable, we're probably in a lot of trouble, but we could still show the web page with the listing. Perhaps we just hide the shopping cart or replace it with an icon saying "Be Back Soon!"

With a single-process monolithic application, we don't have many decisions to make. System health is, to an extent, binary in this context—the process is either up or down. But with a microservice architecture, we need to consider a much more nuanced situation. The right thing to do in any situation is often not a technical decision. We might

know what is technically possible when the shopping cart is down, but unless we understand the business context we won't understand what action we should be taking. For example, perhaps we close the entire site, still allow people to browse the catalog of items, or replace the part of the UI containing the cart control with a phone number for placing an order. But for every customer-facing interface that uses multiple microservices, or every microservice that depends on multiple downstream collaborators, you need to ask yourself, "What happens if this is down?" and know what to do.

By thinking about the criticality of each of our capabilities in terms of our cross-functional requirements, we'll be much better positioned to know what we can do. Now let's consider some things we can do from a technical point of view to make sure that when failure occurs, we can handle it gracefully.

Stability Patterns

There are a few patterns we can make use of to ensure that if something does go wrong, it doesn't cause nasty ripple-out effects. It is essential you understand these ideas, and you should strongly consider making use of them in your system to ensure that one bad citizen doesn't bring the whole world crashing down around your ears. In a moment, we'll take a look at a few key safety measures you should consider, but before we do, I'd like to share a brief story to outline the sort of thing that can go wrong.

Many years ago, I was a technical lead on a project for AdvertCorp. AdvertCorp (the company name and details are changed to protect the innocent!) provided online classified ads through a very popular website. The website itself handled fairly high volumes and generated a good deal of income for the business. The project I was working on was tasked with consolidating a number of existing services that were used to deliver similar functionality for different types of advertisements. Existing functionality for the different types of ads was slowly being migrated into the new system we were building, with a number of different types of ads still served up from older services. To make this transition transparent to the end customer, we intercepted all calls to the different types of ads in our new system, diverting them to the old systems where required, as outlined in Figure 12-1. This is actually an example of a *strangler fig pattern*, which we discussed briefly in "Useful Decompositional Patterns".

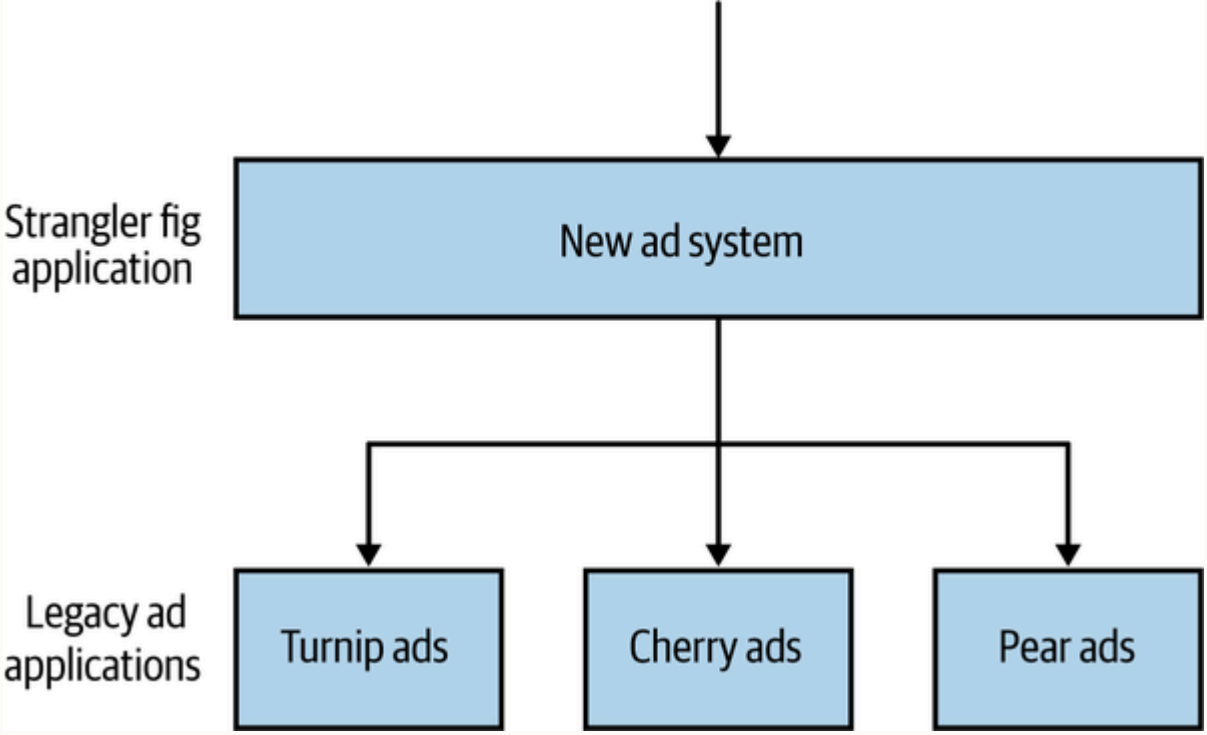


Figure 12-1. A strangler fig pattern being used to direct calls to older legacy systems

We had just moved over the highest-volume and biggest-earning product to the new system, but much of the rest of the ads were still being served by a number of older applications. In terms of both the number of searches and the money made by these applications, there was a very long tail—many of these older applications received small amounts of traffic and generated small amounts of revenue. The new system had been live for a while and was behaving very well, handling a not insignificant load. At that time we must have been handling around 6,000–7,000 requests per second during peak, and although most of that was very heavily cached by reverse proxies sitting in front of our application servers, the searches for products (the most important aspect of the site) were mostly uncached and required a full server round trip.

One morning, just before we hit our daily lunchtime peak, the system started behaving slowly, and then it started failing. We had some level of monitoring on our new core application, enough to tell us that each of our application nodes was hitting a 100% CPU spike, well above the normal levels even at peak. In a short period of time, the entire site went down.

We managed to track down the culprit and bring the site back up. It turned out to be one of the downstream ad systems, which for the sake of this anonymous case study we'll say was responsible for turnip-related ads. The turnip ad service, one of the oldest and least actively maintained services, had started responding very slowly. Responding very slowly is one of the worst failure modes you can experience. If a system is just not there, you find out pretty quickly. When it's just *slow*, you end up waiting around for a while before giving up—that process of waiting around can slow the entire system down, cause resource contention, and, as happened in our case, result in a cascading failure. But

whatever the cause of the failure, we had created a system that was vulnerable to a downstream issue cascading to cause a system-wide failure. A downstream service, over which we had little control, was able to take down our whole system.

While one team looked at the problems with the turnip system, the rest of us started looking at what had gone wrong in our application. We found a few problems, which are outlined in Figure 12-2. We were using an HTTP connection pool to handle our downstream connections. The threads in the pool itself had time-outs configured for how long they would wait when making the downstream HTTP call, which is good. The problem was that the workers were all taking a while to time out due to the slow downstream service. While they were waiting, more requests went to the pool asking for worker threads. With no workers available, these requests themselves hung. It turned out the connection pool library we were using did have a time-out for waiting for workers, but this was *disabled by default*! This led to a huge buildup of blocked threads. Our application normally had 40 concurrent connections at any given time. In the space of five minutes, this situation caused us to peak at around 800 connections, bringing the system down.

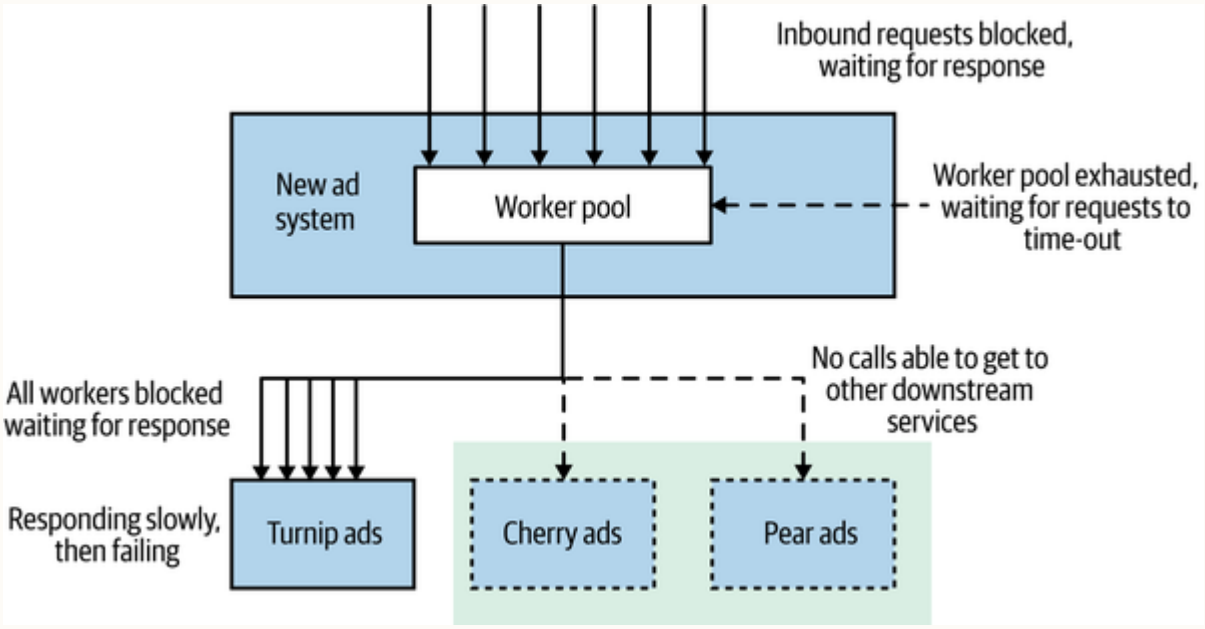


Figure 12-2. An outline of the issues that caused the outage

What was worse was that the downstream service we were talking to represented functionality that less than 5% of our customer base used, and it generated even less revenue than that. When you get down to it, we discovered the hard way that systems that just act slow are *much* harder to deal with than systems that just fail fast. In a distributed system, latency kills.

Even if we'd had the time-outs on the pool set correctly, we were also sharing a single HTTP connection pool for all outbound requests. This meant that one slow downstream

service could exhaust the number of available workers all by itself, even if everything else was healthy. Lastly, it was clear due to the frequent time-outs and errors that the downstream service in question wasn't healthy, but despite this we kept sending traffic its way. In our situation, this meant we were actually making a bad situation worse, as the downstream service had no chance to recover. We ended up implementing three fixes to avoid this happening again: getting our *time-outs* right, implementing *bulkheads* to separate out different connection pools, and implementing a *circuit breaker* to avoid sending calls to an unhealthy system in the first place.

Time-Outs

Time-outs are easy to overlook, but in a distributed system they are important to get right. How long can I wait before I should give up on a call to a downstream service? If you wait too long to decide that a call has failed, you can slow the whole system down. Time out too quickly, and you'll consider a call that might have worked as failed. Have no time-outs at all, and a downstream service being down could hang your whole system.

In the case of AdvertCorp, we had two time-out related issues. Firstly, we had a missing time-out on the HTTP request pool, meaning that when asking for a worker to make a downstream HTTP request, the request thread would block forever until a worker became available. Secondly, when we finally had an HTTP worker available to make a request to the turnip ad system, we were waiting way too long before giving up on the call. So as Figure 12-3 shows, we needed to add one new time-out and change an existing one.

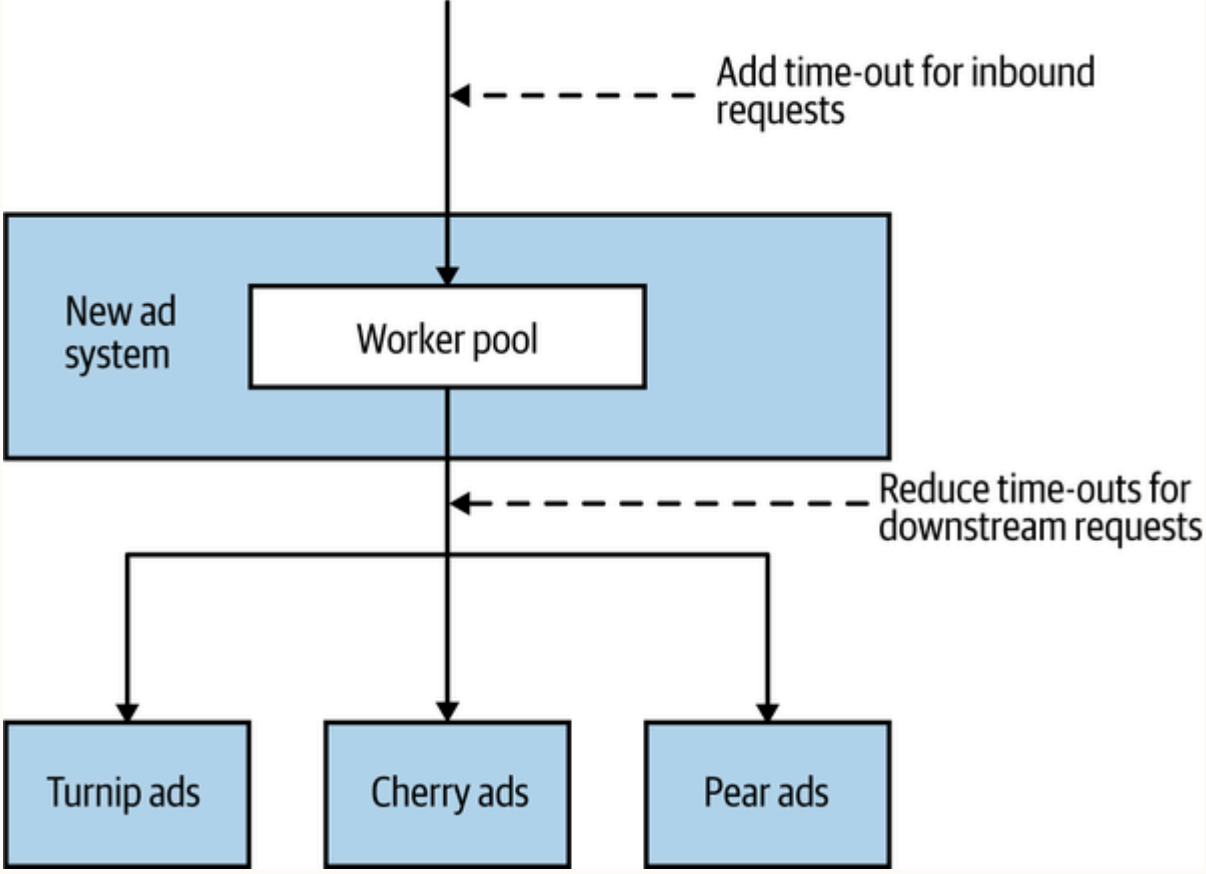


Figure 12-3. Changing time-outs on AdvertCorp system

The time-outs for the downstream HTTP requests had been set to 30 seconds—so we would wait 30 seconds for a response from the turnip system before giving up. The issue is that in the wider context that this call was being made, waiting that long made no sense. The turnip-related ads were being requested as a result of one of our users looking at our website using a browser. Even back when this happened, no one waited 30 seconds for a page to load. Think about what happens if a web page doesn't load after 5, 10, or perhaps 15 seconds. What do you do? You refresh the page. So we were waiting 30 seconds for the turnip ad system to respond, but well before that the original request was no longer valid as the user had just refreshed, causing an additional inbound request. This in turn caused another request to come in for the ad system, and so it went.

When looking at the normal behavior of the turnip ad system, we could see that we would normally expect a response in much less than a second, so waiting for 30 seconds was overkill. Furthermore, we had a target to render a page to the user within 4–6 seconds. Based on this, we made the time-out much more aggressive, setting it to 1 second. We also put a 1 second time-out on waiting for an HTTP worker to be available. This meant that in the worst case, we would expect to wait around 2 seconds for information from the turnip system.

TIP

Time-outs are incredibly useful. Put time-outs on all out-of-process calls, and pick a default time-out for everything. Log when time-outs occur, look at what happens, and change them accordingly. Look at "normal" healthy response times for your downstream services, and use that to guide where you set the time-out threshold.

Setting a time-out for a single service call might not be enough. What happens if this time-out is happening as part of a wider set of operations that you might want to give up on even before the time-out occurs? In the case of AdvertCorp, for example, there is no point waiting for the latest turnip prices if there is a good chance the user has already given up asking. In such a situation, it can make sense to have a time-out for the overall operation and to give up if this time-out is exceeded. For that to work, the current time left for the operation would need to be passed downstream. For example, if the overall operation to render a page had to complete within 1,000 ms, and by the time we made the call to the downstream turnip ad service 300 ms had already passed, we would then need to make sure we waited no longer than 700 ms for the rest of the calls to complete.

WARNING

Don't just think about the time-out for a single service call; also think about a time-out for the overall operation, and abort the operation if this overall time-out budget is exceeded.

Retries

Some issues with downstream calls are temporary. Packets can get misplaced, or gateways can have an odd spike in load, causing a time-out. Often, retrying the call can make a lot of sense. Coming back to what we just talked about, how often have you refreshed a web page that didn't load, only to find the second attempt worked fine? That's a retry in action.

It can be useful to consider what sort of downstream call failures should even be retried. If using a protocol like HTTP, for example, you may get back some useful information in the response codes that can help you determine if a retry is warranted. If you got back a `404 Not Found`, a retry is unlikely to be a useful idea. On the other hand, a `503 Service Unavailable` or a `504 Gateway Time-out` could be considered temporary errors and could justify a retry.

You will likely need to have a delay before retrying. If the initial time-out or error was caused by the fact that the downstream microservice was under load, then bombarding it with additional requests may well be a bad idea.

If you are going to retry, you need to take this into account when considering your time-out threshold. If the time-out threshold for a downstream call is set to 500 ms, but you allow up to three retries with one second between each retry, then you could end up waiting for up to 3.5 seconds before giving up. As mentioned earlier, having a budget for how long an operation is allowed to take can be a useful idea—you might not decide to do the third (or even second) retry if you've already exceeded the overall time-out budget. On the other hand, if this is happening as part of a non-user-facing operation, waiting longer to get something done might be totally acceptable.

Bulkheads

In *Release It!*,[4] Michael Nygard introduces the concept of a *bulkhead* as a way to isolate yourself from failure. In shipping, a bulkhead is a part of the ship that can be sealed off to protect the rest of the ship. So if the ship springs a leak, you can close the bulkhead doors. You lose part of the ship, but the rest of it remains intact.

In software architecture terms, there are lots of different bulkheads we can consider. Returning to my own experience with AdvertCorp, we actually missed the chance to implement a bulkhead with regard to the downstream calls. We should have used different connection pools for each downstream connection. That way, if one connection pool got exhausted, the other connections wouldn't be impacted, as we see in Figure 12-4.

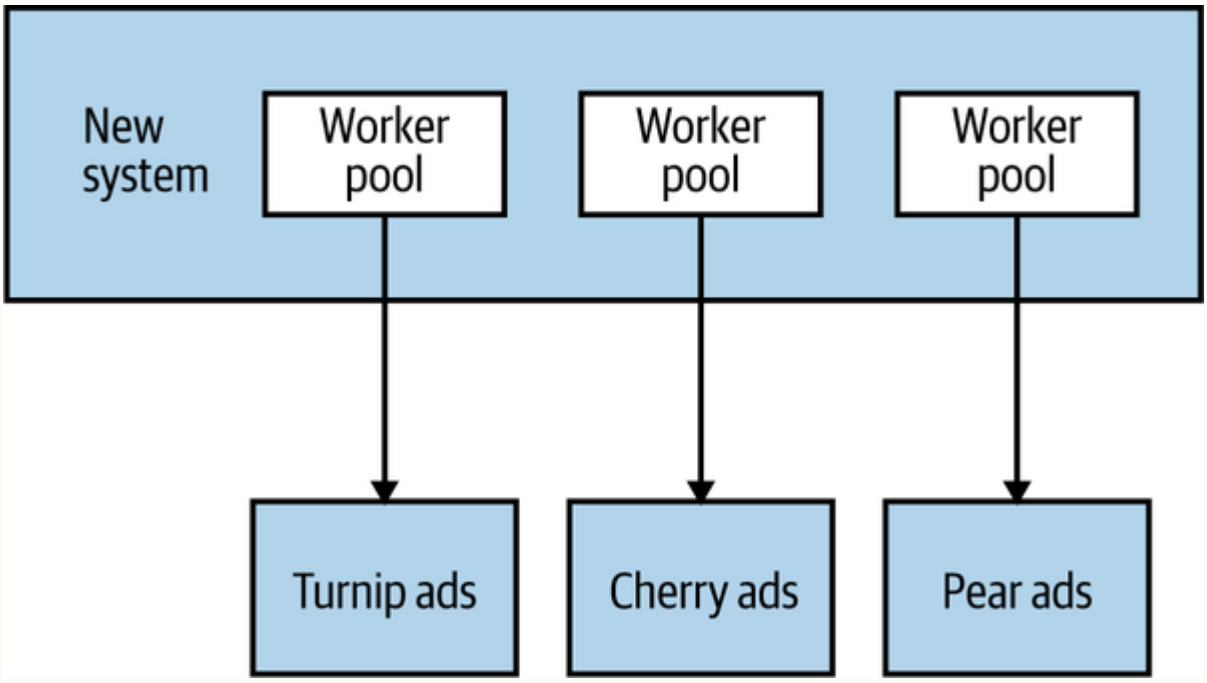


Figure 12-4. Using a connection pool per downstream service to provide *bulkheads*

Separation of concerns can also be a way to implement bulkheads. By teasing apart functionality into separate microservices, we reduce the chance of an outage in one area affecting another.

Look at all the aspects of your system that can go wrong, both inside your microservices and between them. Do you have bulkheads in place? I'd suggest starting with separate connection pools for each downstream connection, at the very least. You may want to go further, however, and consider using circuit breakers too, which we'll look at in a moment.

In many ways, bulkheads are the most important of the patterns we've looked at so far. Time-outs and circuit breakers help you free up resources when they are becoming constrained, but bulkheads can ensure they don't become constrained in the first place. They can also give you the ability to reject requests in certain conditions to ensure that resources don't become even more saturated; this is known as *load shedding*. Sometimes rejecting a request is the best way to stop an important system from becoming overwhelmed and being a bottleneck for multiple upstream services.

Circuit Breakers

In your own home, circuit breakers exist to protect your electrical devices from spikes in power. If a spike occurs, the circuit breaker gets blown, protecting your expensive home appliances. You can also manually disable a circuit breaker to cut the power to part of your home, allowing you to work safely on the electrical system. In another pattern from *Release It!*, Nygard shows how the same idea can work wonders as a protection mechanism for our software.

We can think of our circuit breakers as an automatic mechanism to seal a bulkhead, not only to protect the consumer from the downstream problem but also to potentially protect the downstream service from more calls that may be having an adverse impact. Given the perils of cascading failure, I'd recommend mandating circuit breakers for all your synchronous downstream calls. You don't have to write your own, either—in the years since I wrote the first edition of this book, circuit breaker implementations have become widely available.

Coming back to AdvertCorp, consider the problem we had with the turnip system responding very slowly before eventually returning an error. Even if we'd gotten the time-outs right, we'd be waiting a long time before we got the error. And then we'd try it again the next time a request came in, and wait. It was bad enough that the downstream service was malfunctioning, but it was slowing down the entire system too.

With a circuit breaker, after a certain number of requests to the downstream resource have failed (due either to error or to a time-out), the circuit breaker is blown. All further requests that go through that circuit breaker fail fast while the breaker is in its blown (open) state,[5] as you can see in Figure 12-5. After a certain period of time, the client sends a few requests through to see if the downstream service has recovered, and if it gets enough healthy responses it resets the circuit breaker.

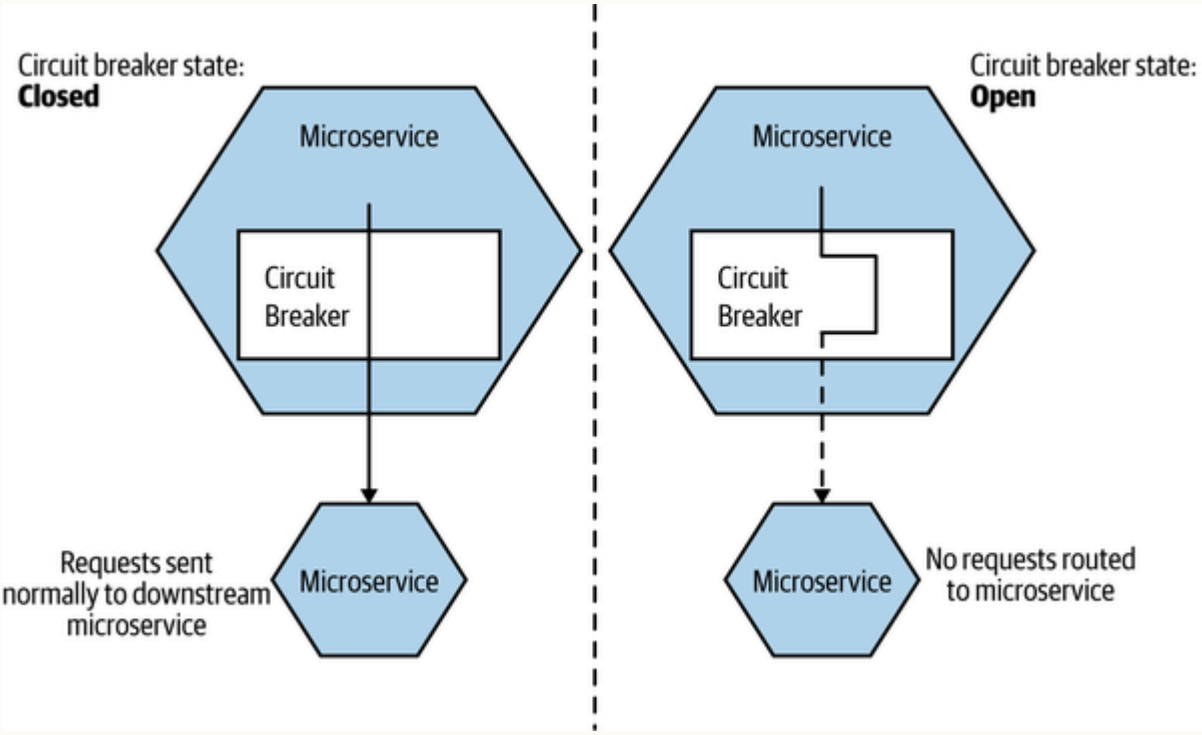


Figure 12-5. An overview of circuit breakers

How you implement a circuit breaker depends on what a "failed" request means, but when I've implemented them for HTTP connections, I've commonly taken failure to mean either a time-out or a subset of the 5XX HTTP return codes. In this way, when a downstream resource is timing out or returning errors, after a certain threshold is reached we automatically stop sending traffic and start failing fast. And we can automatically start again when things are healthy.

Getting the settings right can be a little tricky. You don't want to blow the circuit breaker too readily, nor do you want to take too long to blow it. Likewise, you really want to make sure that the downstream service is healthy again before sending traffic. As with time-outs, I'd pick some sensible defaults and stick with them everywhere, and then change them for specific cases.

While the circuit breaker is blown, you have some options. One is to queue up the requests and retry them later on. For some use cases, this might be appropriate, especially if you're carrying out some work as part of an asynchronous job. If this call is being made as part of a synchronous call chain, however, it is probably better to fail fast. This could mean propagating an error up the call chain, or a more subtle degrading of functionality.

In the case of AdvertCorp, we wrapped downstream calls to the legacy systems with circuit breakers, as Figure 12-6 shows. When these circuit breakers blew, we programmatically updated the website to show that we couldn't currently show ads for, say, turnips. We kept the rest of the website working and clearly communicated to the customers that there was an issue restricted to one part of our product, all in a fully automated way.

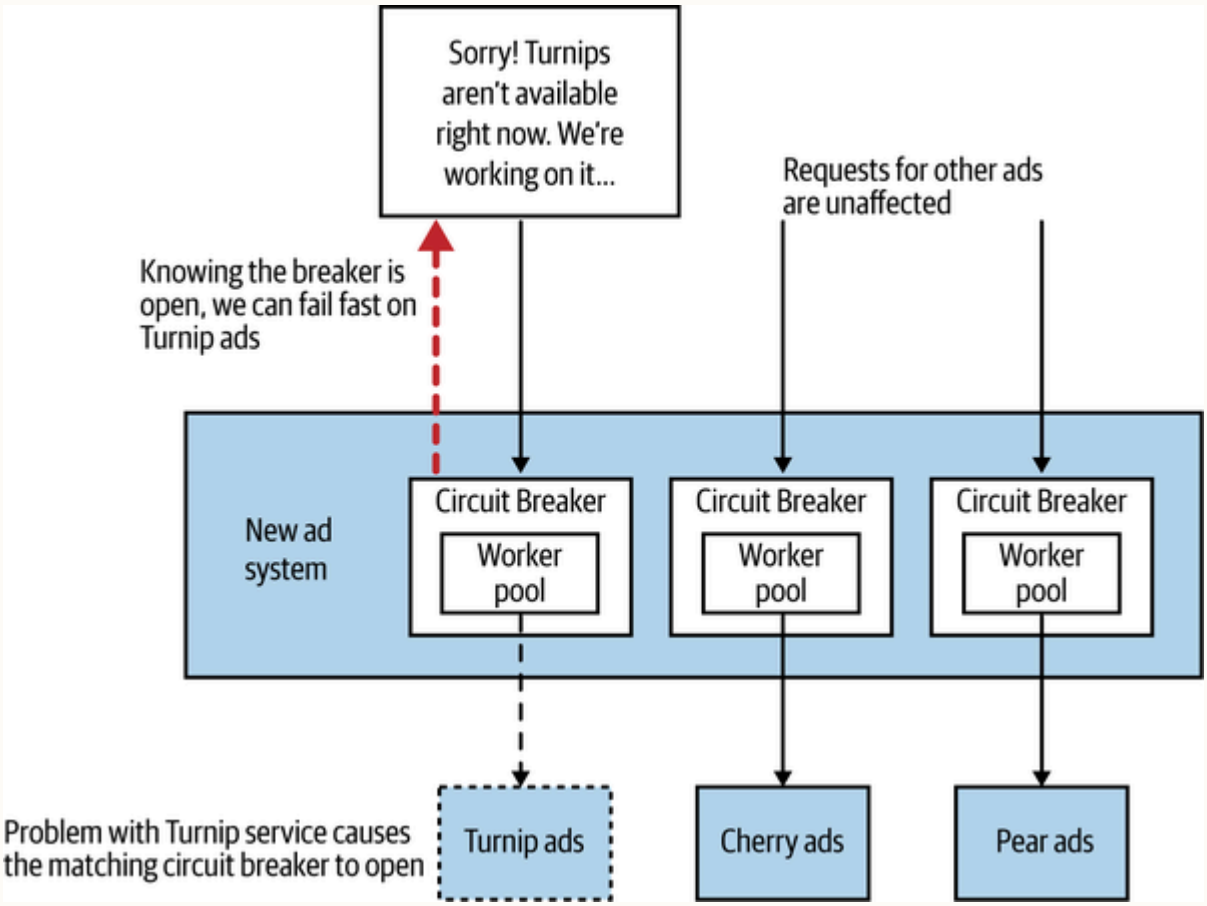


Figure 12-6. Adding circuit breakers to AdvertCorp

We were able to scope our circuit breakers so that we had one for each of the downstream legacy systems—this lined up well with the fact that we had decided to have different request worker pools for each downstream service.

If we have this mechanism in place (as with the circuit breakers in our home), we could use them manually to make it safer to do our work. For example, if we wanted to take a microservice down as part of routine maintenance, we could manually open all the circuit breakers of the upstream consumers so they fail fast while the microservice is offline. Once it's back, we can close the circuit breakers, and everything should go back to normal. Scripting the process to manually open and close a circuit breaker as part of an automated deployment process could be a sensible next step.

Circuit breakers help our application fail fast—and failing fast is always better than failing slow. The circuit breakers allow us to fail before wasting valuable time (and resources) waiting for an unhealthy downstream microservice to respond. Rather than

waiting until we try to use the downstream microservice to fail, we could check the status of our circuit breakers earlier. If a microservice we will rely on as part of an operation is currently unavailable, we can abort the operation before we even start.

Isolation

The more one microservice depends on another microservice being available, the more the health of one impacts the ability of the other to do its job. If we can use technology that allows a downstream server to be offline, for example, through the use of middleware or some other type of call buffering system, upstream microservices are less likely to be affected by outages, planned or unplanned, of downstream microservices.

There is another benefit to increasing isolation between services. When services are isolated from each other, much less coordination is needed between service owners. The less coordination needed between teams, the more autonomy those teams have, as they are able to operate and evolve their services more freely.

Isolation also applies in terms of how we move from the logical to the physical. Consider two microservices that appear to be entirely isolated from one another. They don't communicate with each other in any way. A problem with one of them shouldn't impact the other, right? But what if both microservices are running on the same host, and one of the microservices starts using up all the CPU, causing that host to have issues?

Consider another example. Two microservices each have their own, logically isolated database. But both databases are deployed onto the same database infrastructure. A failure in that database infrastructure would impact both microservices.

When we consider how we want to deploy our microservices, we also want to strive to ensure a degree of failure isolation to avoid problems like this. For example, ensuring that microservices are running on independent hosts with their own ring-fenced operating system and computing resources is a sensible step—this is what we achieve when we run microservice instances in their own virtual machine or container. This kind of isolation, though, can have a cost.

We can isolate our microservices from each other more effectively by running them on different machines. This means we need more infrastructure, and tooling to manage that infrastructure. This has a direct cost and can also add to the complexity of our system exposing new avenues of potential failure. Each microservice could have its own totally dedicated database infrastructure, but that is more infrastructure to manage. We could

use middleware to provide temporal decoupling between two microservices, but now we have a broker to worry about.

Isolation, like so many of the other techniques we have looked at, can help improve the robustness of our applications, but it's rare that it does so for free. Deciding on the acceptable trade-offs around isolation versus cost and increased complexity, like so many other things, can be vital.

Redundancy

Having more of something can be a great way to improve the robustness of a component. Having more than one person who knows how the production database works seems sensible, in case someone leaves the company or is away on leave. Having more than one microservice instance makes sense as it allows you to tolerate the failure of one of those instances and still have a chance of delivering the required functionality.

Working out how much redundancy you need, and where, will depend on how well you understand potential failure modes of each component, the impact of that functionality being unavailable, and the cost of adding the redundancy.

On AWS, for example, you do not get an SLA for the uptime of a single EC2 (virtual machine) instance. You have to work on the assumption that it can and will die on you. So it makes sense to have more than one. But going further, EC2 instances are deployed into availability zones (virtual data centers), and you also have no guarantees regarding the availability of a single availability zone, meaning you'd want that second instance to be on a *different* availability zone to spread the risk.

Having more copies of something can help when it comes to implementing redundancy, but it can also be beneficial when it comes to scaling our applications to handle increased load. In the next chapter we'll look at examples of system scaling, and see how scaling for redundancy or scaling for load can differ.

Middleware

In "Message Brokers", we looked at the role of middleware in the form of message brokers to help with implementing both request-response and event-based interactions. One of the useful properties of most message brokers is their ability to provide guaranteed delivery. You send a message to a downstream party, and the broker guarantees to deliver it, with some caveats that we explored earlier. Internally, to provide this guarantee, the message broker software will have to implement things like retries and time-

outs on your behalf—the same sorts of operations are being carried out as you would have to do yourself, but they are being done in software written by experts with a deep focus on this sort of thing. Having smart people do work for you is often a good idea.

Now, in the case of our specific example with AdvertCorp, using middleware to manage request-response communication with the downstream turnip system might not actually have helped much. We would still not be getting responses back to our customers. The one potential benefit would be that we'd be relieving resource contention on our own system, but this would just shift to an increasing number of pending requests being held in the broker. Worse still, many of these requests asking for the latest turnip prices might relate to user requests that are no longer valid.

An alternative could be to instead invert the interaction and use middleware to have the turnip system broadcast the last turnip ads, and we could then consume them. But if the downstream turnip system had a problem, we still wouldn't be able to help the customer looking for the best turnip prices.

So using middleware like message brokers to help offload some robustness concerns can be useful, but not in every situation.

Idempotency

In *idempotent* operations, the outcome doesn't change after the first application, even if the operation is subsequently applied multiple times. If operations are idempotent, we can repeat the call multiple times without adverse impact. This is very useful when we want to replay messages that we aren't sure have been processed, a common way of recovering from error.

Let's consider a simple call to add some points as a result of one of our customers placing an order. We might make a call with the sort of payload shown in Example 12-1.

Example 12-1. Crediting points to an account

```
<credit>
  <amount>100</amount>
  <forAccount>1234</account>
</credit>
```

If this call is received multiple times, we would add 100 points multiple times. As it stands, therefore, this call is not idempotent. With a bit more information, though, we

allow the points bank to make this call idempotent, as shown in Example 12-2.

Example 12-2. Adding more information to the points credit to make it idempotent

```
<credit>
 <amount>100</amount>
 <forAccount>1234</account>
 <reason>
  <forPurchase>4567</forPurchase>
 </reason>
</credit>
```

We know that this credit relates to a specific order, `4567`. Assuming that we could receive only one credit for a given order, we could apply this credit again without increasing the overall number of points.

This mechanism works just as well with event-based collaboration and can be especially useful if you have multiple instances of the same type of service subscribing to events. Even if we store which events have been processed, with some forms of asynchronous message delivery there may be small windows in which two workers can see the same message. By processing the events in an idempotent manner, we ensure this won't cause us any issues.

Some people get quite caught up with this concept and assume it means that subsequent calls with the same parameters can't have *any* impact, which then leaves us in an interesting position. We really would still like to record the fact that a call was received in our logs, for example. We want to record the response time of the call and collect this data for monitoring. The key point here is that it is the underlying business operation that we are considering idempotent, not the entire state of the system.

Some of the HTTP verbs, such as GET and PUT, are defined in the HTTP specification to be idempotent, but for that to be the case, they rely on your service handling these calls in an idempotent manner. If you start making these verbs nonidempotent, but callers think they can safely execute them repeatedly, you may get yourself into a mess. Remember, just because you're using HTTP as an underlying protocol doesn't mean you get everything for free!

Spreading Your Risk

One way to scale for resilience is to ensure that you don't put all your eggs in one basket. A simplistic example of this is making sure that you don't have multiple services on one host, where an outage would impact multiple services. But let's consider what *host* means. In most situations nowadays, a "host" is actually a virtual concept. So what if I have all of my services on different hosts, but all those hosts are actually virtual hosts, running on the same physical box? If that box goes down, I could lose multiple services. Some virtualization platforms enable you to ensure that your hosts are distributed across multiple different physical boxes to reduce the chance of this happening.

For internal virtualization platforms, it is a common practice to have the virtual machine's root partition mapped to a single SAN (storage area network). If that SAN goes down, it can take down all connected VMs. SANs are big, expensive, and designed not to fail. That said, I have had big, expensive SANs fail on me at least twice in the last 10 years, and each time the results were fairly serious.

Another common form of separation to reduce failure is to ensure that not all your services are running in a single rack in the data center, or that your services are distributed across more than one data center. If you're using an underlying service provider, it is important to know if an SLA is offered and to plan accordingly. If you need to ensure your services are down for no more than four hours every quarter, but your hosting provider can only guarantee a maximum downtime of eight hours per quarter, you have to change the SLA or else come up with an alternative solution.

AWS, for example, is split into regions, which you can think of as distinct clouds. Each region is in turn split into two or more availability zones, as we discussed earlier. These availability zones are AWS's equivalent of a data center. It is essential to have services distributed across multiple availability zones, as AWS does not offer any guarantees about the availability of a single node, or even an entire availability zone. For its compute service, it offers only a 99.95% uptime over a given monthly period of the region as a whole, so you'll want to distribute your workloads across multiple availability zones inside a single region. For some people, this isn't good enough, and they run their services across multiple regions as well.

It should be noted, of course, that because providers give you an SLA "guarantee," they will tend to limit their liability! If them missing their targets results in you losing customers and a large amount of money, you might find yourself searching through contracts to see if you can claw anything back from them. Therefore, I would strongly sug-

gest you understand the impact of a supplier failing in its obligations to you, and work out if you need to have a plan B (or C) in your pocket. More than one client I've worked with has had a disaster recovery hosting platform with a different supplier, for example, to ensure they weren't too vulnerable to the mistakes of one company.

CAP Theorem

We'd like to have it all, but unfortunately we know we can't. And when it comes to distributed systems like those we build using microservice architectures, we even have a mathematical proof that tells us we can't. You may well have heard about the CAP theorem, especially in discussions about the merits of various different types of data stores. At its heart it tells us that in a distributed system, we have three things we can trade off against each other: *consistency*, *availability*, and *partition tolerance*. Specifically, the theorem tells us that we get to keep two in a failure mode.

Consistency is the system characteristic by which we will get the same answer if we go to multiple nodes. Availability means that every request receives a response. Partition tolerance is the system's ability to handle the fact that communication between its parts is sometimes impossible.

Since Eric Brewer published his original conjecture, the idea has gained a mathematical proof. I'm not going to dive into the math of the proof itself, as not only is this not that sort of book, but I can also guarantee that I would get it wrong. Instead, let's use some worked examples that will help us understand that under it all, the CAP theorem is a distillation of a very logical set of reasoning.

Let's imagine that our `Inventory` microservice is deployed across two separate data centers, as shown in Figure 12-7. Backing our service instance in each data center is a database, and these two databases talk to each other to try to synchronize data between them. Reads and writes are done via the local database node, and replication is used to synchronize the data between the nodes.

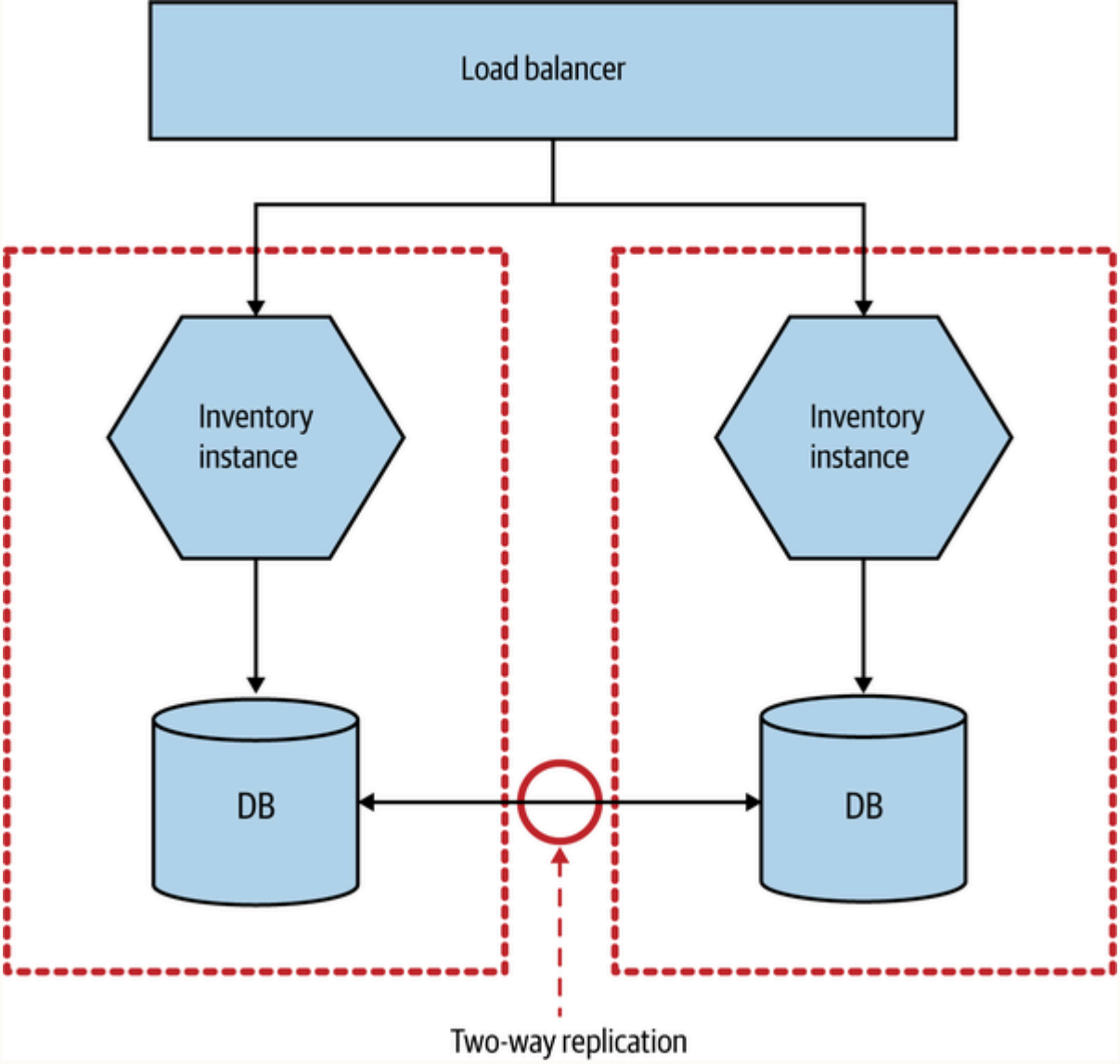


Figure 12-7. Using multiprimary replication to share data between two database nodes

Now let's think about what happens when something fails. Imagine that something as simple as the network link between the two data centers stops working. The synchronization at this point fails. Writes made to the primary database in DC1 will not propagate to DC2, and vice versa. Most databases that support these setups also support some sort of queuing technique to ensure that we can recover from this afterward, but what happens in the meantime?

Sacrificing Consistency

Let's assume that we don't shut the `Inventory` microservice down entirely. If I make a change now to the data in DC1, the database in DC2 doesn't see it. This means any requests made to our inventory node in DC2 see potentially stale data. In other words, our system is still *available* in that both nodes are able to serve requests, and we have kept the system running despite the *partition*, but we have lost *consistency*; we don't get to keep all three traits. This is often called an *AP* system, because of its availability and partition tolerance.

During this partition, if we keep accepting writes, then we accept the fact that at some point in the future they have to be resynchronized. The longer the partition lasts, the more difficult this resynchronization can become.

The reality is that even if we don't have a network failure between our database nodes, replication of data is not instantaneous. As touched on earlier, systems that are happy to cede consistency to keep partition tolerance and availability are said to be *eventually consistent*; that is, we expect at some point in the future that all nodes will see the updated data, but it won't happen at once, so we have to live with the possibility that users see old data.

Sacrificing Availability

What happens if we need to keep consistency and want to drop something else instead? Well, to keep consistency, each database node needs to know the copy of the data it has is the same as the other database node. Now in the partition, if the database nodes can't talk to each other, they cannot coordinate to ensure consistency. We are unable to guarantee consistency, so our only option is to refuse to respond to the request. In other words, we have sacrificed availability. Our system is consistent and partition tolerant, or CP. In this mode our service would have to work out how to degrade functionality until the partition is healed and the database nodes can be resynchronized.

Consistency across multiple nodes is really hard. There are few things (perhaps nothing) harder in distributed systems. Think about it for a moment. Imagine I want to read a record from the local database node. How do I know it is up to date? I have to go and ask the other node. But I also have to ask that database node to not allow it to be updated while the read completes; in other words, I need to initiate a transactional read across multiple database nodes to ensure consistency. But in general people don't do transactional reads, do they? Because transactional reads are slow. They require locks. A read can block an entire system.

As we've already discussed, distributed systems have to expect failure. Consider our transactional read across a set of consistent nodes. I ask a remote node to lock a given record while the read is initiated. I complete the read and ask the remote node to release its lock, but now I can't talk to it. What happens now? Locks are really hard to get right even in a single process system and are significantly more difficult to implement well in a distributed system.

Remember when we talked about distributed transactions in Chapter 6? The core reason they are challenging is because of this problem with ensuring consistency across multi-

ple nodes.

Getting multinode consistency right is so hard that I would strongly, *strongly* suggest that if you need it, don't try to invent it yourself. Instead, pick a data store or lock service that offers these characteristics. Consul, for example, which we discussed in "Dynamic Service Registries", implements a strongly consistent key-value store designed to share configuration between multiple nodes. Along with "Friends don't let friends write their own crypto" should go "Friends don't let friends write their own distributed consistent data store." If you think you need to write your own CP data store, read all the papers on the subject first, then get a PhD, and then look forward to spending a few years getting it wrong. Meanwhile, I'll be using something off the shelf that does it for me, or more likely trying *really hard* to build eventually consistent AP systems instead.

Sacrificing Partition Tolerance?

We get to pick two, right? So we've got our eventually consistent AP system. We have our consistent but hard-to-build-and-scale CP system. Why not a CA system? Well, how can we sacrifice partition tolerance? If our system has no partition tolerance, it can't run over a network. In other words, it needs to be a single process operating locally. CA systems don't exist in distributed systems.

AP or CP?

Which is right, AP or CP? Well, the reality is that *it depends*. As the people building the system, we know the trade-off exists. We know that AP systems scale more easily and are simpler to build, and we know that a CP system will require more work due to the challenges in supporting distributed consistency. But we may not understand the business impact of this trade-off. For our inventory system, if a record is out of date by five minutes, is that OK? If the answer is yes, an AP system might be the answer. But what about the balance held for a customer in a bank? Can that be out of date? Without knowing the context in which the operation is being used, we can't know the right thing to do. Knowing about the CAP theorem helps you understand that this trade-off exists and what questions to ask.

It's Not All or Nothing

Our system as a whole doesn't need to be either AP or CP. Our catalog for MusicCorp could be AP, as we don't worry too much about a stale record. But we might decide that

our inventory service needs to be CP, as we don't want to sell a customer something we don't have and then have to apologize later.

But individual services don't even need to be CP or AP.

Let's think about our `Points Balance` microservice, where we store records of how many loyalty points our customers have built up. We could decide that we don't care if the balance we show for a customer is stale, but that when it comes to updating a balance we need it to be consistent to ensure that customers don't use more points than they have available. Is this microservice CP, or AP, or both? Really, what we have done is push the trade-offs around the CAP theorem down to individual microservice capabilities.

Another complexity is that neither consistency nor availability is all or nothing. Many systems allow us a far more nuanced trade-off. For example, with Cassandra I can make different trade-offs for individual calls. So if I need strict consistency, I can perform a read that blocks until all replicas have responded, confirming the value is consistent, or until a specific quorum of replicas have responded, or even just a single node. Obviously, if I block waiting for all replicas to report back and one of them is unavailable, I'll be blocking for a long time. On the other hand, if I wanted my read to respond as quickly as possible, I might wait to hear back from only a single node—in which case there is a possibility that this might be an inconsistent view of my data.

You'll often see posts about people "beating" the CAP theorem. They haven't. What they have done is create a system in which some capabilities are CP, and some are AP. The mathematical proof behind the CAP theorem holds.

And the Real World

Much of what we've talked about is the electronic world—bits and bytes stored in memory. We talk about consistency in an almost childlike fashion; we imagine that within the scope of the system we have created, we can stop the world and have it all make sense. And yet so much of what we build is just a reflection of the real world, and we don't get to control that, do we?

Let's revisit our inventory system. This maps to real-world, physical items. We keep a count in our system of how many albums we have in MusicCorp's warehouses. At the start of the day we had 100 copies of *Give Blood* by the Brakes. We sold one. Now we have 99 copies. Easy, right? But what happens if when the order is being sent out, someone knocks a copy of the album onto the floor and it gets stepped on and broken? What

happens now? Our systems say there are 99 copies on the shelf, but there are actually only 98.

What if we made our inventory system AP instead, and we occasionally had to contact a user later on and tell him that one of his items is actually out of stock? Would that be the worst thing in the world? It would certainly be much easier to build and scale it and ensure it is correct.

We have to recognize that no matter how consistent our systems might be in and of themselves, they cannot know everything that happens, especially when we're keeping records of the real world. This is one of the main reasons that AP systems end up being the right call in many situations. Aside from the complexity of building CP systems, they can't fix all our problems anyway.

ANTIFRAGILITY

In the first edition, I talked about the concept of antifragile, as popularized by Nassim Taleb. This concept describes how systems actually benefit from failure and disorder and was highlighted as being inspiration for how some parts of Netflix operated, specifically with respect to concepts like chaos engineering. When looking more broadly at the concept of resiliency, though, we realize that antifragility is just a subset of the concept of resiliency. When we consider the concepts of graceful extensibility and sustained adaptability, which we introduced earlier, this becomes clear.

I think that when antifragility became, briefly, a hyped concept in IT, it did so against a backdrop of us thinking narrowly about resiliency—where we thought only about robustness, and maybe about rebound, but ignored the rest. With the field of resilience engineering now gaining more recognition and traction, it seems appropriate to move beyond the term *antifragile*, while still ensuring that we highlight some of the ideas behind it that are well and truly part of resiliency as a whole.

Chaos Engineering

Since the first edition of this book, another technique that has gained more attention is *chaos engineering*. Named for practices used at Netflix, it can be a useful approach to help improve your resiliency—either in terms of ensuring your systems are as *robust* as you think they are, or else as part of an approach toward the *sustained adaptability* of your system.

Originally inspired by work that Netflix was doing internally, the term *chaos engineering* struggles somewhat due to confusing explanations of what it means. For many, it means "run a tool on my software and see what it says," something arguably not helped by the fact that many of the most vocal proponents of chaos engineering are often themselves selling tools to run on your software to do, well, chaos engineering.

Getting a clear definition of what chaos engineering means to its practitioners is difficult. The best definition (at least to my mind) that I've come across is this:

> *Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production.*
>
> —*Principles of Chaos Engineering*

Now the word *system* here is doing a lot of work. Some will view this narrowly as the software and hardware components. But in the context of resiliency engineering, it's important that we see the *system* as being the entirety of the people, the processes, the culture, and yes, the software and infrastructure that go into creating our product. This means we should view chaos engineering more broadly than just "let's turn off some machines and see what happens."

Game Days

Well before chaos engineering had its name, people would run Game Day exercises to test people's preparedness for certain events. Planned in advance but ideally launched in surprise (for the participants), this gives you the chance to test your people and processes in the wake of a realistic but fictional situation. During my time at Google, this was a fairly common occurrence for various systems, and I certainly think that many organizations could benefit from having these sorts of exercises regularly. Google goes beyond simple tests to mimic server failure, and as part of its DiRT (Disaster Recovery Test) exercises it has simulated large-scale disasters such as earthquakes.[6]

Game Days can be used to probe suspected sources of weakness in the system. In his book, *Learning Chaos Engineering*,[7] Russ Miles shares the example of a Game Day exercise he facilitated that had been designed to examine, in part, the overreliance on a single staff member named Bob. For the Game Day, Bob was sequestered in a room and unable to help the team during the simulated outage. Bob was observing, though, and ended up having to step in when the team, in its attempt to fix issues with the "fake" system, ended up logging into production by mistake and was in the process of destroying pro-

duction data. One can only assume that lots of lessons were learned in the aftermath of that exercise.

Production Experiments

The scale at which Netflix operates is well known, as is the fact that Netflix is based entirely on the AWS infrastructure. These two factors mean that it has to embrace failure well. Netflix realized that planning for failure and actually *knowing* your software will handle that failure when it occurs are two different things. To that end, Netflix actually *incites* failure to ensure that its systems are tolerant of failure by running tools on its systems.

The most famous of these tools is Chaos Monkey, which during certain hours of the day turns off random machines in production. Knowing that this can and will happen in production means that the developers who create the systems really have to be prepared for it. Chaos Monkey is just one part of Netflix's Simian Army of failure bots. Chaos Gorilla is used to take out an entire availability zone (the AWS equivalent of a data center), whereas Latency Monkey simulates slow network connectivity between machines. For many, the ultimate test of whether your system really is robust might be unleashing your very own Simian Army on your production infrastructure.

From Robustness to Beyond

Applied in its narrowest form, chaos engineering could be a useful activity in terms of improving the robustness of our application. Remember, robustness in the context of resilience engineering means the extent to which our system can handle expected problems. Netflix knew it couldn't rely on any given virtual machine being available in its production environment, so it built Chaos Monkey to ensure that its system could survive this *expected* issue.

If you make use of chaos engineering tooling as part of an approach to continually question the resilience of your system, however, it can have far greater applicability. Using tools in this space to help answer the "what if" questions you might have, continually questioning your understanding, can have a much bigger impact. The Chaos Toolkit is an open source project to help you run experiments on your system, and it has proved very popular. Reliably, the company founded by the creators of the Chaos Toolkit, offers a broader range of tools to help with chaos engineering in general, although perhaps the best-known vendor in this space is Gremlin.

Just remember, running a chaos engineering tool doesn't make you resilient.

Blame

When things go wrong, there are lots of ways we can handle it. Obviously, in the immediate aftermath, our focus is on getting things back up and running, which is sensible. After that, all too often, come the recriminations. There is a default position to look for something or someone to blame. The concept of "root cause analysis" implies there *is* a root cause. It's surprising how often we want that root cause to be a human.

A few years ago, back when I was working in Australia, Telstra, the main telco (and previous monopoly) had a major outage that impacted both voice and telephony services. This incident was especially problematic due to the scope and duration of the outage. Australia has many very isolated rural communities, and outages like this tend to be especially serious. Almost immediately after the outage, Telstra's COO put out a statement that made it clear they knew exactly what had caused the problem:[8]

> *"We took that node down, unfortunately the individual that was managing that issue did not follow the correct procedure, and he reconnected the customers to the malfunctioning node, rather than transferring them to the nine other redundant nodes that he should have transferred people to," Ms. McKenzie told reporters on Tuesday afternoon.*
>
> *"We apologise right across our customer base. This is an embarrassing human error."*

So firstly, note that this statement came out hours after the outage. And yet Telstra had already unpacked what must be a vastly complex system to know exactly what was to blame—one person. Now, if it's true that one person making a mistake can really bring an entire telco to its knees, you'd think that would say more about the telco than the individual. In addition, Telstra clearly signaled to its staff at the time that it was very happy to point fingers and apportion blame.[9]

The problem with blaming people in the aftermath of incidents like this is that what starts as a short-term passing of the buck ends up creating a culture of fear, where people will be unwilling to come forward to tell you when things go wrong. As a result, you will lose the opportunity to learn from failure, setting yourself up for the same problems happening again. Creating an organization in which people have the safety to admit when mistakes are made is essential in creating a learning culture, and in turn can go a long way toward creating an organization that is able to create more robust software, quite aside from the obvious benefits of creating a happier place to work in.

Coming back to Telstra, with the cause of the blame clearly established in the in-depth investigation that was carried out mere hours after the nationwide outage, we'd clearly expect no subsequent outages, right? Unfortunately, Telstra suffered a string of subsequent outages. More human error? Perhaps Telstra thought so—in the wake of the series of incidents, the COO resigned.

For a more informed take on how to create an organization where you can get the best out of mistakes and create a nicer environment for your staff as well, John Allspaw's "Blameless Post-Mortems and a Just Culture" is a great jumping-off point.[10]

Ultimately, as I've already highlighted numerous times in this chapter, *resiliency* requires a questioning mind—a drive to constantly examine the weaknesses in our system. This requires a culture of learning, and often the best learning can come in the wake of an incident. It's therefore vital that you ensure that when the worst happens, you do your best to create an environment in which you can maximize the information you gather in the aftermath to reduce the chances of it happening again.

Summary

As our software becomes more vital to the lives of our users, the drive toward improving the resiliency of the software we create increases. As we've seen in this chapter, though, we cannot achieve resiliency *just* by thinking about our software and infrastructure; we also have to think about our people, processes, and organizations. In this chapter we looked at the four core concepts of resiliency, as described by David Woods:

Robustness

: The ability to absorb expected perturbation

Rebound

: The ability to recover after a traumatic event

Graceful extensibility

: How well we deal with a situation that is unexpected

Sustained adaptability

: The ability to continually adapt to changing environments, stakeholders, and demands

Looking more narrowly at microservices, they give us a host of ways in which we can improve the *robustness* of our systems. But this improved robustness is not free—you still have to decide which options to use. Key stability patterns like circuit breakers, timeouts, redundancy, isolation, idempotency, and the like are all tools at your disposal, but you have to decide when and where to use them. Beyond these narrow concepts, though, we also need to be constantly on the lookout for what we don't know.

You also have to work out how much resiliency you want—and this is nearly always something defined by the users and business owners of your system. As a technologist, you can be in charge of how things are done, but knowing what resiliency is needed will require good, frequent close communication with users and product owners.

Let's come back to David Woods's quote from earlier in the chapter, which we used when discussing sustained adaptability:

> *No matter how good we have done before, no matter how successful we've been, the future could be different, and we might not be well adapted. We might be precarious and fragile in the face of that new future.*

Asking the same questions over and over again doesn't help you understand if you are prepared for the uncertain future. You don't know what you don't know—adopting an approach in which you are constantly learning, constantly questioning, is key to building resiliency.

One type of stability pattern we looked at, redundancy, can be very effective. This idea segues quite nicely into our next chapter, where we'll look at different ways to scale our microservices, which as well as helping us handle more load can also be an effective way to help us implement redundancy in our systems, and therefore also improve the robustness of our systems.

1 David D. Woods, "Four Concepts for Resilience and the Implications for the Future of Resilience Engineering," *Reliability Engineering & System Safety* 141 (September 2015): 5–9, doi.org/10.1016/j.ress.2015.03.018.

2 Jason Bloomberg, "Innovation: The Flip Side of Resilience," *Forbes*, September 23, 2014, *https://oreil.ly/avSmU*.

3 See "Google Uncloaks Once-Secret Server" by Stephen Shankland for more information on this, including an interesting overview of why Google thinks this approach can be superior to traditional UPS systems.

4 Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. (Raleigh: Pragmatic Bookshelf, 2018).

5 The terminology of an "open" breaker, meaning requests can't flow, can be confusing, but it comes from electrical circuits. When the breaker is "open," the circuit is broken and current cannot flow. Closing the breaker allows the circuit to be completed and current to flow once again.

6 Kripa Krishnan, "Weathering the Unexpected," acmqueue 10, no. 9 (2012), *https://oreil.ly/BCSQ7*.

7 Russ Miles, *Learning Chaos Engineering* (Sebastopol: O'Reilly, 2019).

8 Kate Aubusson and Tim Biggs, "Major Telstra Mobile Outage Hits Nationwide, with Calls and Data Affected," *Sydney Morning Herald*, February 9, 2016, *https://oreil.ly/4cBcy*.

9 I wrote more about the Telstra incident at the time—"Telstra, Human Error and Blame Culture," *https://oreil.ly/OXgUQ*. When I wrote that blog post, I neglected to realize that the company I worked for had Telstra as a client; my then-employer actually handled the situation extremely well, even though it did make for an uncomfortable few hours when some of my comments got picked up by the national press.

10 John Allspaw, "Blameless Post-Mortems and a Just Culture," Code as Craft (blog), Etsy, May 22, 2012, *https://oreil.ly/7LzmL*.